

Project Title: Stock Market Data Processing using PySpark and MapReduce

1. Processing objective & analytical question

The objective of the project was to demonstrate how large-scale financial dataset can be processed using Apache Spark and MapReduce to produce meaningful analytical results.

The analysis focused on the following research question:

How do stock-market sectors differ in trading activity, price behaviour, returns and volatility, and what insights can distributed PySpark processing provide from large-scale intraday stock-market dataset?

The processing workflow was designed to answer four main questions:

1. Which sectors demonstrate the greatest trading activity?
2. Which stocks and sectors demonstrate the greatest historical return volatility?
3. How do average returns differ between sectors?
4. How can Spark's distributed processing model transform a large number of raw observations into relatively small analytical summaries?

The processing pipeline was designed as follows:

Raw CSV files → Mapping/Transformation → Return calculation → Shuffle/Grouping → Reduction/Aggregation → Sector summaries → Visualisation and interpretation

Pandas was only used after Spark had reduced the dataset to relatively small result tables for visualisation.

2. Mapping /Transformation

The processing dataset consisted of complete CSV files selected from the larger working dataset. This meant some stock tickers were dropped to select the 3.28GB of files while keeping the original proportionality of the stocks representation from the raw dataset.

The CSV files were identified using the code below:

```
processing_files = list(PROCESSING_ROOT.rglob("*.csv"))
```

The resulting file list was then passed explicitly to Spark:

```
df_raw = (spark.read.option("header", True).option("inferSchema", True)
          .csv([str(f) for f in processing_files]))
```

The original dataset structure contains the sector and ticker information in the directory and filename.

For example: 15_min_interval_stocks/Basic_Materials/AU.csv

Represents: sector = Basic_Materials and ticker = AU

The source path was therefore retained in Spark:

```
df = df_raw.withColumn("source_file", F.input_file_name())
```

The ticker and sector were then extracted using regular expressions:

```
df = (df.withColumn("ticker", F.regexp_extract("source_file", r"([^\w]+\)\.csv$", 1))
      .withColumn("sector", F.regexp_extract("source_file", r"[\w]([^\w]+\)[\w]([^\w]+\)\.csv$", 1)))
```

This transformation converts the filesystem structure into analytical attributes. This is important because the sector and ticker become the keys used during the subsequent grouping and reduction stages.

Group 2

MIT805: Big Data Project (2026) - Part 2

The raw CSV fields were converted to appropriate Spark data types:

```
df = (df.withColumn("timestamp", F.to_timestamp("timestamp"))
      .withColumn("open", F.col("open").cast("double"))
      .withColumn("high", F.col("high").cast("double"))
      .withColumn("low", F.col("low").cast("double"))
      .withColumn("close", F.col("close").cast("double"))
      .withColumn("volume", F.col("volume").cast("double")))
```

This transformation is important because numerical calculations such as averages, standard deviations and returns require numerical data types. The analysis removed records that could not contribute reliably to the calculations.

The cleaning criteria were:

```
df_clean = df.filter(F.col("timestamp").isNotNull()
                    & F.col("sector").isNotNull() & F.col("ticker").isNotNull() & F.col("close").isNotNull()
                    & F.col("volume").isNotNull() & (F.col("close") > 0) & (F.col("volume") >= 0))
```

The filtering was performed using Spark rather than Pandas, ensuring that the full processing dataset did not have to be transferred into local memory. Returns were required to compare price movements between observations rather than simply comparing nominal stock prices.

The previous closing price was obtained using:

```
df_returns = df_clean.withColumn("Previous_close", F.lag("close").over(window_spec))
```

Returns were required to compare price movements between observations rather than simply comparing nominal stock prices.

The corresponding Spark transformation to calculate the return was:

```
df_returns = df_returns.withColumn("return", F.when(F.col("previous_close").isNotNull()
            & (F.col("previous_close") != 0), (F.col("close") - F.col("previous_close")) /
            F.col("previous_close")))
```

The first observation for each ticker has no previous closing price and therefore produces a null return. This transformation is particularly important because return volatility, rather than raw price variation, is used later as the primary measure of historical price risk.

A further derived measure was dollar trading volume, implemented in Spark as:

```
df_returns = df_returns.withColumn("dollar_volume", F.col("close") * F.col("volume"))
```

This provides an estimate of the monetary value represented by trading activity in each observation. It complements share volume because two stocks can have similar numbers of shares traded but substantially different monetary values due to differences in their share prices.

3. Shuffle / Grouping

The next stage required observations to be grouped according to their analytical keys. The first major grouping operation was: **stock_grouped = df_returns.groupBy("Sector", "Ticker")**

The purpose of this operation was to ensure that observations belonging to the same stock could be processed together.

For example, all observations for: Technology + AAPL must contribute to the same stock-level aggregate. Because Spark DataFrames are partitioned, observations for a particular ticker may initially exist across different partitions. Spark therefore needs to redistribute data according to the grouping key. This redistribution represents the Shuffle/Grouping stage of the MapReduce conceptual model. The shuffle is an important part of distributed processing because it involves data movement between partitions. It can therefore become one of the computationally expensive stages of a large Spark workload.

Group 2

MIT805: Big Data Project (2026) - Part 2

The formatted execution plan was examined using: `stock_grouped.count().explain("formatted")` and later: `sector_summary.explain("formatted")` which provides evidence of Spark's physical execution strategy, including exchange and aggregation operations.

4. Reduction /Aggregation

4.1. Stock-level aggregation

After grouping the records by sector and ticker, Spark reduced the individual observations into stock-level summary statistics.

The following aggregation was used:

```
stock_summary = (df_returns.groupBy("sector", "ticker")
    .agg(F.count("*").alias("observations"),
        F.avg("close").alias("average_close"), F.min("close").alias("minimum_close"),
        F.max("close").alias("maximum_close"), F.stddev("close").alias("price_stddev"),
        F.avg("volume").alias("average_volume"), F.sum("volume").alias("total_volume"),
        F.avg("dollar_volume").alias("average_dollar_volume"),
        F.sum("dollar_volume").alias("total_dollar_volume"),\
        F.avg("return").alias("average_return"), F.stddev("return").alias("return_volatility")))
```

This operation reduces potentially thousands or millions of observations belonging to one stock into a single summary record.

4.2. Sector-level aggregation

A second reduction was performed at sector level:

```
sector_summary = (df_returns.groupBy("sector")
    .agg(F.countDistinct("ticker").alias("number_of_stocks"),
        F.count("*").alias("observations"), F.avg("close").alias("average_close"),
        F.avg("volume").alias("average_volume"), F.sum("volume").alias("total_volume"),
        F.avg("dollar_volume").alias("average_dollar_volume"),
        F.sum("dollar_volume").alias("total_dollar_volume"),
        F.avg("return").alias("average_return"),
        F.stddev("return").alias("return_volatility")).orderBy(F.desc("total_volume")))
```

This second reduction provided the main dataset used to answer the research question.

sector	number_of_stocks	observations	average_close	average_volume	total_volume
Consumer_Discretionary	176	12240801	213.53433426477508	56943.023097998244	6.97028214081E11
Technology	100	7131412	596.5041818293945	88730.75494516373	6.32775570585E11
Financials	204	13398066	57.63680775232787	32627.436604059123	4.37144549032E11
Healthcare	153	8637654	5151.59500710241	50148.540493055174	4.33165741384E11
Industrials	76	6883893	181.4073282951126	39484.691082647565	2.71808388551E11
Basic_Materials	12	1480048	21.082025216006578	147141.86022277657	2.17777015939E11
Energy	20	2416126	1679.4550241917239	89171.3675305013	2.15449259546E11
Utilities	29	2399434	22.92722021076624	50183.461594275985	1.20411903987E11
Consumer_Staples	23	1705103	66.47755977339743	58211.505206430345	9.9256612162E10
Real_Estate	36	2953852	27.609587413282778	27415.77300724613	8.0982135929E10
Telecommunications	16	1274365	28487.943357222277	26648.76539374512	3.3960253911E10
Other	39	918111	90.01603671887256	26263.291302467784	2.4112616641E10

Figure 1: Sector-level Spark results

5. Spark execution & DAG

5.1. Spark execution model

The processing was executed using PySpark in local mode:

```
spark = (SparkSession.builder.appName("MIT805_StockMarket_Analysis")
    .master("local[*]").config("spark.sql.shuffle.partitions", "8").getOrCreate())
```

The `local[*]` configuration allows Spark to use the available 4 CPU cores on the local machine. It is important to distinguish this from a distributed multi-node cluster. The analysis demonstrates the Spark distributed processing architecture, partitioning, shuffling, task execution and DAG planning, but the actual execution environment was a local Spark instance.

Group 2

MIT805: Big Data Project (2026) - Part 2

5.2. DAG representation

The overall Spark execution can be represented conceptually as:

CSV File Scan → Schema / Projection → Sector + Ticker Mapping → Filtering → Window / Sorting → Return Calculation → Dollar Volume Mapping → Exchange / Shuffle → Stock-level Aggregate → Exchange / Shuffle → Sector-level Aggregate → Final Output.

The DAG illustrates that Spark does not simply execute a traditional Map → Reduce sequence.

Instead, Spark creates a logical and physical execution plan and divides the work into stages and tasks. Operations that require redistribution of data create shuffle boundaries.

The stock-level **groupBy()** is therefore associated with an Exchange/Shuffle operation, while the subsequent aggregation performs reduction.

5.3. Evidence from Spark execution plan

The execution plan was inspected using: **sector_summary.explain("formatted")**

```
sector_summary.explain("formatted")

*** == Physical Plan ==
AdaptiveSparkPlan (16)
+- Sort (15)
   +- Exchange (14)
      +- HashAggregate (13)
         +- Exchange (12)
            +- HashAggregate (11)
               +- HashAggregate (10)
                  +- HashAggregate (9)
                     +- Project (8)
                        +- Window (7)
                           +- Sort (6)
                              +- Exchange (5)
                                 +- Project (4)
                                    +- Filter (3)
                                       +- Project (2)
                                          +- Scan csv (1)
```

Figure 2: Spark formatted physical execution plan

Figure 2 shows the Spark physical execution plan containing the scan, transformations, exchange/shuffle and aggregation stages.

6. Results & technical interpretation

The Spark processing produced three principal analytical outputs:

- stock-level summaries;
- sector-level summaries;
- ranked lists of highly active and highly volatile stocks.

The top-volume stocks were obtained using:

```
top_volume_stocks = (stock_summary.orderBy(F.desc("total_volume")).limit(10))
```

Similarly, the highest-volatility stocks were obtained using:

```
top_volatility_stocks = (stock_summary.filter(F.col("return_volatility").isNotNull())
    .orderBy(F.desc("return_volatility")).limit(10))
```

These operations were executed by Spark before the results were collected for presentation.

6.1. Overall processing results

```
=====
Processing dataset size: 3.05 GB
Processing files: 884
Raw observations: 61,438,865
Clean observations: 61,438,865
Stocks analysed: 884
Sectors analysed: 12
```

Figure 3: Overall processing Metadata

```
=====
Most active sector: Consumer_Discretionary
Highest total volume: 697,028,214,081
Most volatile sector: Healthcare
Return volatility: 2.151978
Highest average return sector: Healthcare
Average return: 0.002068
=====
```

Figure 4: Key Results

These values provide evidence of the scale of the workload and demonstrate the reduction from the original observation-level dataset to a relatively small analytical result.

Group 2
MIT805: Big Data Project (2026) - Part 2

6.2. Trading activity

The sector-level total_volume metric was used to identify the most active sector. The Most active sector is **Consumer_Discretionary** with a Total volume of **697028214081.0**

The most active sector represents the largest aggregate number of shares traded in the selected processing dataset.

However, this result must be interpreted in the context of sector representation. Because sectors can contain different numbers of securities and securities can have different historical coverage, a higher total volume does not necessarily mean that every stock in that sector is more liquid.

For this reason, the analysis also calculated average stock-level volume.

6.3. Volatility

Return volatility was measured using the standard deviation of interval returns. The Most volatile sector is **Healthcare** with a Return of **2.151978**. A higher standard deviation indicates greater variation in historical returns. From a financial perspective, this can be interpreted as greater historical short-term price uncertainty. However, volatility does not distinguish between positive and negative movements; it measures the magnitude of variation around the average return.

6.4. Average returns

The sector-level average return was used to compare historical price movement. The Highest average-return sector is **Healthcare** and an Average return of **0.002068**. A higher average return indicates stronger average historical movement over the observations included in the calculation. This result should not be interpreted as a prediction of future performance. The analysis is descriptive and can be affected by the historical period, the number of observations, irregular intervals and the composition of each sector.

6.5. Technical interpretation

The most important technical result is the transformation of a 3.28 GB raw processing dataset into compact analytical tables using Spark. The raw dataset contains individual observations, while the final sector summary contains only one row per sector.

3.28 GB raw data → Spark transformations → Window calculation → Shuffle → Stock aggregation → Sector aggregation → Small analytical dataset

This demonstrates the value of distributed processing. The computationally intensive work is performed before the data is collected for visualisation. As a result, only a small amount of data needs to be transferred into Pandas.

7. Visualization

The visualisations were generated from the aggregated Spark results rather than the complete raw dataset. The sector results were collected only after Spark had completed the aggregation: **sector_pd = sector_summary.toPandas()**. This was appropriate because the sector summary is substantially smaller than the original 3.28 GB processing dataset.

7.1. Trading volume by sector

The results show substantial differences in aggregate trading activity between sectors. **Consumer_Discretionary** recorded the highest total trading volume, while **Other** recorded the lowest. This difference reflects the combined effect of the number of securities represented, their historical coverage and their trading activity.

Group 2

MIT805: Big Data Project (2026) - Part 2

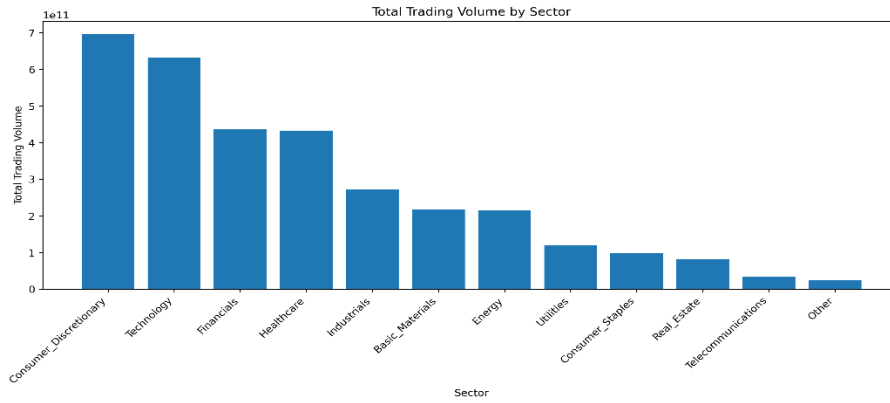


Figure 5: Total Trading Volume by Sector

7.2. Return volatility by sector

The sector-level volatility results show that historical return variability differs across the stock-market sectors. **Healthcare** recorded the highest return volatility, indicating greater variation in its observed returns during the analysed period. **Real_Estate** recorded the lowest volatility.

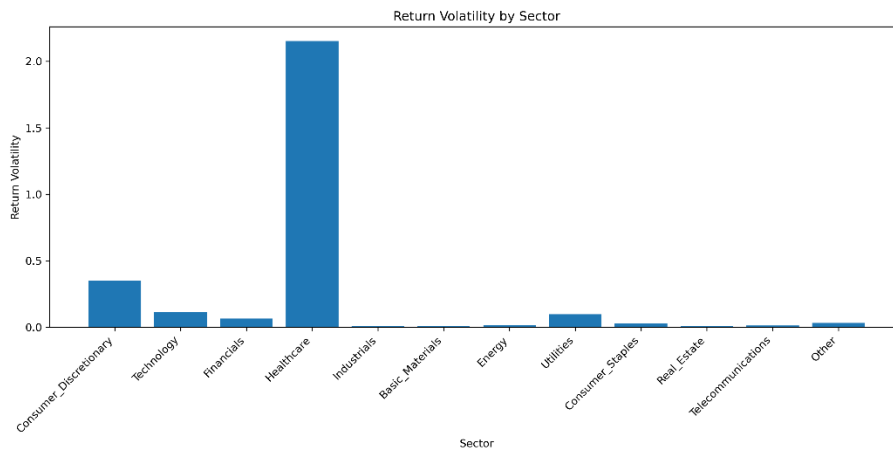


Figure 6: Return volatility by sector

7.3. Average return by sector

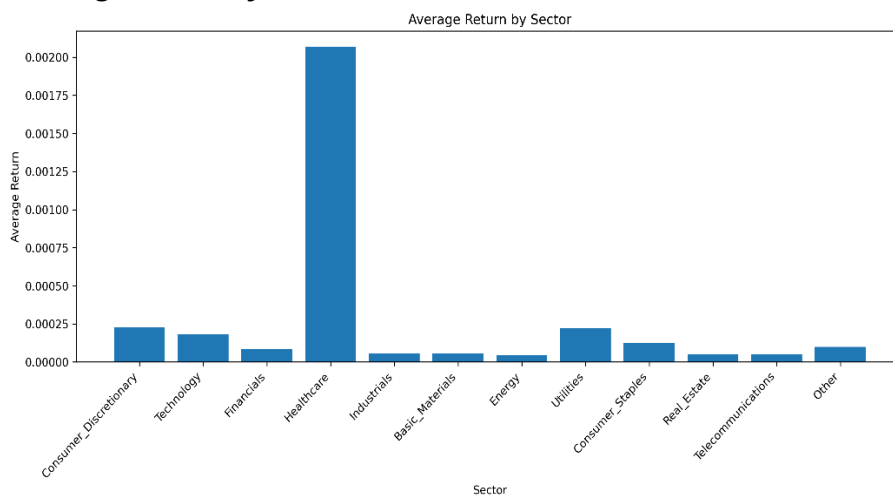


Figure 7: Average Return by Sector

The average-return comparison demonstrates that historical price movement was not uniform across sectors. **Healthcare** recorded the highest average return, while **Energy** recorded the lowest. These values describe historical observations and should not be interpreted as forecasts of future performance.

7.4. Top 10 stocks by trading volume



Figure 8: Top 10 Stocks by Total Trading Volume

Trading activity was concentrated among a subset of securities. The top ten stocks contributed substantially to the observed trading volume within the processing dataset. This demonstrates why stock-level analysis is important: sector-level totals can conceal large differences between individual securities.

8. Business/societal interpretation & value

The analysis demonstrates how Big Data processing can support financial decision-making by transforming large volumes of raw market observations into interpretable stock and sector indicators.

- The sector-level return and volatility measures can help investors compare the historical characteristics of different sectors. For example, a portfolio manager could use historical volatility as one input when assessing sector exposure and diversification.
- The volatility analysis has particular relevance to financial risk management. Financial institutions may need to monitor large numbers of securities simultaneously. A Spark-based processing architecture allows large volumes of historical market observations to be transformed into risk indicators at stock and sector level.
- Trading volume provides an indication of market activity. The combination of total volume, average volume, dollar volume, stock-level rankings, sector-level rankings, provides a more informative picture than using a single measure. A financial institution could use similar processing to identify securities with unusually high or low trading activity.

Limitations

- **Dataset sampling:** The processing dataset is 3.28 GB, compared with the larger working dataset of more than 12 GB. Although complete ticker files were selected and the stock representation was kept proportional, the processing dataset does not contain every stock in the working dataset.
- **Temporal irregularity:** Although the directory is labelled as 15-minute data, the exploratory analysis identified differences in observation intervals. Therefore, the calculated return represents the change between consecutive available observations and should not automatically be interpreted as a standardised 15-minute return.